■ SUJET D'ÉVALUATION : SUITE DE TESTS SPRING BOOT

Microservice « Pokémon Management API » — Master 2 I

1. Contexte & Objectifs Pédagogiques

Dans le cadre du SI de la Ligue Pokémon, vous devez concevoir et tester un microservice Spring Boot. L'objectif principal est de développer une suite de **tests unitaires et ciblés** sur les 3 couches de l'architecture (**Repository, Service, Controller**), en respectant l'isolation des composants et la structure **Arrange - Act - Assert (AAA)**.

2. Modèle de Données & Règle Métier Implémentée

A. Entités JPA & Relation

1. **Dresseur** : id (Long), nom (String), region (String), pokemons (Relation @OneToMany vers Pokemon).
2. **Pokemon** : id (Long), nom (String), type (String : 'Feu', 'Eau', 'Électrik'...), niveau (int), dresseur (Relation @One vers Dresseur).

B. Règle Métier : Calcul du Niveau du Dresseur

Dans la couche Service (DresseurService), implémentez la méthode calculerNiveauDresseur(Long dresseurId) selon la règle métier suivante :

FORMULE DE CALCUL DU NIVEAU DRESSEUR

• **Base de calcul :** Somme exacte des niveaux (niveau) de tous les Pokémon possédés.
• **Bonus Diversité de Types :** Si le dresseur possède au moins 3 types distincts de Pokémon, ajout d'un bonus de **+20 points**.
• **Bonus Pokémon Élite :** Chaque Pokémon de niveau ≥ 50 apporte un bonus supplémentaire de **+10 points**. • **Cas limite (0 Pokémon) :** Si le dresseur n'a aucun Pokémon, son niveau est égal à **1**.
• **Endpoint :** Endpoint dresseurs/{id} retourne un DresseurDto (nomDresseur, nombrePokemons, niveauDresseur). On doit verifier si le dresseur existe.

3. Travail à Réaliser : Suite de Tests par Couche

1. Repository (DresseurRepositoryTest)	@DataJpaTest Base H2 data.sql	• Tester la persistance. • Initialiser le jeu de données de test via src/test/resources/data.sql. • Valider les résultats avec des assertions fluides **assertEquals**.
2. Service (DresseurServiceTest)	Mock	• Tester la règle métier de calcul du niveau en **isolation totale**. • Simuler les retours du repository via when(...).thenReturn(...). • Tester le cas nominal, le cas 0 Pokémon, le cumul des bonus
3. Controller (DresseurControllerTest)	@WebMvcTest MockMvc	• Simuler l'endpoint REST GET /dresseurs/{id}. • Valider les codes de statut HTTP (200 OK, 201 Dresseur n'existe pas).

4. Directives de Qualité & Consignes de Rendu

• **Convention de Nommage des Tests :** Respecter le format should_[comportement_attendu]_when_[condition]() (ex: should_apply_diversity_bonus_when_dresseur_has_3_different_types()).
• **Structure AAA :** Organiser le corps de chaque méthode de test en trois blocs : **// Arrange**, **// Act**, et **// Assert**.

Livraisons Attendues (Code Source & Vidéo)

1. **Dépôt Git / Code Source :** Application Spring Boot complète et compilable. La commande mvn install doit s'exécuter avec un succès de 100%.
2. **Vidéo de Présentation Professionnelle (10 minutes MAXIMUM) :**